



Indian Institute of Technology Indore

Science and Technology Council

IITISOC 2026



Mid-Way Evaluation Report

Quantum RL Agent for F1 Race Strategy Optimization

TEAM DETAILS:

Leader: Mayank Gupta (250001047)

Member1: Mayank Jha (250004023)

Member2: Chetna Mundra (250001024)

Member3: Piyushi (250001055)

Submission Date: 30-06-2026

NOTE: This report is submitted as part of the IITISoC 2026 Mid Evaluation (QC × ML Domain). All implementation details are derived solely from completed project codebase and documentation (Dated: 28-06-2026).

Contents

1. Problem Statement	3
2. Methodology/Initial Approach	3
2.1 Data Foundation — FastF1 Pipeline	
2.2 Environment Design — Gymnasium Interface	
2.3 Classical RL Baseline — Advanced DQN	
2.4 Quantum RL Agent — VQC-DQN (Planned)	
3. Progress	4
4. Implementation	5
4.1 Data Processing — F1TrackDataLoader	
4.2 Environment — F1StrategyEnv	
4.3 DQN Agent Architecture	
4.4 Training Components	
4.5 Checkpointing and Evaluation	
4.6 Codebase Structure	
5. Results and Analysis	8
5.1 Environment Validation	
5.2 DQN Training Observations	
5.3 DQN Evaluation Outcomes	
5.4 DQN Training and Evaluation Process Updates (Dated: 28-06-2026)	
6. Execution Pipeline	9
7. Final Evaluation Deliverables (Planned)	10
8. GitHub & Workflow Diagram(Planned)	10
9. References and Resources	10

1. Problem Statement

Formula 1 race strategy is one of the most consequential and analytically complex decisions in motorsport. At every lap, a strategist must weigh tyre degradation rates, current race position, proximity to competitors, safety car probability, weather conditions, and regulatory constraints, all while maximizing championship points over the course of a 60–78 lap race.

Traditional strategy tools rely on pre-computed lookup tables, experienced human judgment, and classical simulation engines. These approaches are inherently limited in their ability to generalize across the combinatorial space of race conditions. **Reinforcement Learning (RL)** offers a principled alternative: an agent that learns optimal strategy by interacting with a simulated race environment, without requiring hand-crafted rules for every scenario.

Our project pushes this further by asking a research question at the frontier of **Quantum Computing** and **Machine Learning** (QC × ML): Can a Variational Quantum Circuit (VQC)-based policy act as an effective replacement for the classical neural network in a DQN agent, and if so, does it offer measurable advantages in sample efficiency, convergence, or generalization?

To answer this question rigorously, we must:

- Build a high-fidelity F1 race simulation environment grounded in real historical race data.
- Implement a strong classical DQN baseline with well-documented performance metrics.
- Implement a hybrid quantum-classical DQN agent using a VQC as the policy backbone.
- Benchmark both agents systematically on the same race scenarios, measuring reward convergence, sample efficiency, and strategic output quality.

2. Methodology / Initial Approach

Our project follows a sequential, modular development pipeline with five conceptually distinct layers. Each layer is independently testable and connects directly to the next, ensuring that deficiencies at any stage can be isolated and corrected without destabilizing the full system.

2.1 Data Foundation — FastF1 Pipeline

All race data is sourced from the FastF1 Python library, which provides access to official Formula 1 timing, lap, and telemetry data. We selected three iconic circuits — Monaco (low-speed, tactical), Silverstone (high-energy, high-degradation), and Monza (low-downforce, power-sensitive) — across three seasons (2022, 2023, 2024), yielding nine distinct race scenarios. These circuits were chosen to expose the agent to diverse strategic landscapes.

Raw data is downloaded once and cached at two levels: the raw FastF1 session cache and a processed pickle cache. The processed cache allows training episodes to initialise in under 0.1 seconds, which is critical for iterative RL training where thousands of episode resets occur.

2.2 Environment Design — Gymnasium Interface

The simulation environment, F1StrategyEnv, implements the standard Gymnasium interface (reset, step, render). The environment replays real historical race data lap-by-lap, injecting the agent's decisions as modifications to the simulated driver's race trajectory while the rest of the field follows their recorded historical times. This design grounds the simulation in real-world data without requiring a full multi-agent physics simulation.

2.3 Classical RL Baseline — Advanced DQN

We chose Deep Q-Network (DQN) as the classical RL baseline because it is well-understood, highly tunable, and directly comparable to the VQC variant. Rather than implementing a naive DQN, we

adopted an advanced formulation incorporating Double DQN, Dueling architecture, and Prioritized Experience Replay (PER), making the baseline a competitive benchmark rather than a straw man.

2.4 Quantum RL Agent — VQC-DQN (Planned)

The target quantum architecture replaces the hidden layers of the DQN with a Variational Quantum Circuit implemented in PennyLane. The design uses angle encoding (RY rotations) for state features, ring-topology CNOT entanglement, and 2–4 variational layers of Rot gates. The circuit output — expectation values $\langle Z_i \rangle$ per qubit — feeds into a classical linear readout layer that produces Q-values. Training proceeds via PyTorch autograd through the PennyLane–PyTorch interface.

2.5 Benchmarking and Visualization (Planned)

The final phase will produce a Streamlit-based lap-by-lap replay dashboard using Plotly, enabling side-by-side comparison of QRL, DQN, and real team strategy decisions. Benchmark metrics will include cumulative reward, convergence speed, sample efficiency, and tyre strategy realism.

3. Progress

The project has completed the first three stages of its five-stage pipeline. The table below summarises the current status of each component:

Stage	Component	Tools / Details	Status
1	Data Layer	FastF1 → pandas → parquet / pickle cache	Completed
2	Environment	F1StrategyEnv (Gymnasium) — 15-obs, 6-action	Completed
3	Classical Agent	Advanced DQN (train_dqn.py → checkpoints)	Completed
4	QRL Agent	Hybrid VQC-DQN (PennyLane + PyTorch)	Planned
5	Benchmarking & Dashboard	Metric tables, convergence comparison, Streamlit + Plotly replay UI	Planned

In terms of the project timeline:

- Week 1 (Data Pipeline + Research): Completed. FastF1 extraction, cleaning, and two-level caching implemented for all 9 race/year combinations.
- Week 2 (Environment): Completed. F1StrategyEnv built and validated. Observation space, action space, reward function, safety car handling, wet-race logic, tyre degradation, and driver selection filter all operational.
- Week 3 (Classical DQN): Completed. Advanced DQN with Double DQN, Dueling architecture, PER, action masking, soft target updates, checkpointing, and evaluation workflow implemented and training-ready.
- Week 4 (Quantum RL Agent): In progress. Architecture designed; PennyLane VQC implementation underway.
- Week 5–6 (Benchmarking, Dashboard, Final Report): Planned.

4. Implementation

4.1 Data Processing — F1TrackDataLoader

The F1TrackDataLoader class (my_data.py) transforms raw FastF1 session data into six typed arrays consumed by the environment:

- Base lap time per compound: Computed by filtering to green-flag, accurate, fresh-tyre ($\text{age} \leq 3$) laps and taking the mean of the fastest 20% — capturing front-runner pace. Four progressive fallback levels handle data-sparse compounds.
- Tyre degradation curves: Fitted per compound via a fuel-burn-corrected lap time delta relative to the stint's first lap. Outliers (> 5 s from stint median) are removed. The non-decreasing property is enforced. Default fallback curves are used when data coverage is insufficient.
- Pit stop time loss: Measured per event as pit-in + pit-out lap times minus twice the driver's average clean lap time, then averaged across all stops.
- Historical lap time grid: A driver $\times$ lap sparse matrix used to replay the full field and to compute the field-fastest lap reference each step.
- Safety Car flags: A per-lap integer array (0=green, 1=VSC, 2=full SC) derived from TrackStatus codes.
- Track Wetness flags: Inferred from compound distribution ($\geq 30\%$ of field on wets/inters triggers damp or wet classification) — avoids weather telemetry download.

4.2 Environment — F1StrategyEnv

A single training episode spans one full Grand Prix (60–78 laps). The observation returned at each lap is a 15-dimensional normalised vector:

Idx	Feature	Computation	Meaning
0	Race Progress	current_lap / total_laps	Race progress context (0=start, 1=end)
1	Grid Position	current_position / total_drivers	Running order (0=P1, 1=last)
2	Pace vs Field	$\max(0, 1 - \text{lap_delta} / 10)$	Closeness to fastest lap (1=fastest)
3–7	Tyre Compound	One-hot (5 compounds)	Soft / Med / Hard / Inter / Wet
8	Tyre Wear	deg_now / deg_at_half_race	Relative tyre degradation
9	Gap to Leader	gap_leader / 40 s (capped)	Time gap to P1
10	Gap Ahead	gap_ahead / 10 s (capped)	Proximity to car in front
11	Gap Behind	gap_behind / 10 s (capped)	Defensive urgency
12	Safety Car	0 / 0.5 / 1.0	Green / VSC / Full SC
13	Track Wetness	0 / 0.5 / 1.0	Dry / Damp / Wet
14	2-Compound Rule	0 or 1	Mandatory rule compliance flag

The action space consists of six discrete choices:

Action	Strategy	Effect
0	Stay Out	Continue on current tyres; tyre age increments by 1
1	Pit — Soft	Pit stop; reset to Soft tyres (fastest, least durable)
2	Pit — Medium	Pit stop; reset to Medium tyres (balanced)
3	Pit — Hard	Pit stop; reset to Hard tyres (durable, slowest)
4	Pit — Intermediate	Pit stop; fit Intermediates (damp conditions)
5	Pit — Full Wet	Pit stop; fit full Wet tyres (heavy rain)

The reward function is designed to prioritise long-horizon race outcome:

$$\text{reward} = (\text{position_delta}) * 1.0 + (\text{gap_delta}) * 0.1 - (\text{lap_delta}) * 0.1$$

Terminal bonuses apply at race completion: +25 points maximum for championship points (P1=25, P2=18, ..., P10=1), and a -150 penalty if the mandatory two-compound rule is violated in a dry race. Pit stop penalties are based on real historical pit loss times and are halved under safety car conditions to reflect the strategic advantage of an opportunistic pit.

Key simulation design choices include: driver selection with a $\geq 95\%$ lap-completion retirement filter for episode richness, episode-level seeded randomness for full reproducibility, and lap-accurate weather and safety car replay from historical FastF1 data.

4.3 DQN Agent Architecture

The neural network uses a Dueling DQN architecture. A shared feature extractor (Linear 15→256 with LayerNorm + ReLU, followed by Linear 256→256 with LayerNorm + ReLU) feeds into two separate streams:

- Value stream: Linear(256→128) → ReLU → Linear(128→1), estimating $V(s)$.
- Advantage stream: Linear(256→128) → ReLU → Linear(128→6), estimating $A(s, a)$.

The combined Q-value is $Q(s, a) = V(s) + A(s, a) - \text{mean}(A(s, \cdot))$, which enforces identifiability between the two streams. LayerNorm is used at both shared layers to handle the scale heterogeneity of the 15-feature observation vector.

4.4 Training Components

The training system integrates the following components:

- Double DQN: The online network selects the next action; the target network evaluates it. This decoupling reduces Q-value overestimation.
- Prioritized Experience Replay (PER): Transitions are sampled proportional to $|\text{TD error}|^\alpha$ ($\alpha = 0.6$). Importance-sampling weights (β : 0.4→1.0) correct the resulting bias.
- Soft target updates: Target network parameters are updated each step as $\vartheta_{\text{target}} \rightarrow \tau \cdot \vartheta_{\text{online}} + (1 - \tau) \cdot \vartheta_{\text{target}}$, with $\tau = 0.005$.
- Epsilon-greedy exploration: ϵ decays linearly from 1.0 to 0.05 over 80,000 steps; during evaluation, $\epsilon = 0$.
- Action masking: A domain-aware mask is applied at every action selection step, blocking early pits (lap < 3), immediate re-pit (tyre age ≤ 2), end-of-race pits, same-compound pits, and weather-incompatible tyre choices.

- Gradient clipping: Gradient norm capped at 10.0 to prevent destabilising updates.
- Reward scaling: Raw rewards multiplied by 0.01 before storage, keeping Q-value magnitudes numerically stable.

Current DQN hyperparameter configuration:

Hyperparameter	Value	Purpose
Observation dim	15	15-feature normalised state vector
Action dim	6	Stay out + 5 pit compound choices
Discount factor (γ)	0.99	High future-reward weighting
Learning rate	1e-4	AdamW optimiser step size
Batch size	128	Transitions per training step
Replay capacity	100,000	Max stored transitions
Learning starts	2,000	Warm-up before training begins
Tau (soft update)	0.005	Slow target network follow-rate
Gradient clip	10.0	Prevents exploding gradients
Epsilon start / final	1.0 $\rightarrow$ 0.05	Exploration decay range
Epsilon decay steps	80,000	Duration of exploration decay
Reward scale	0.01	Numerical stability for Q-values
PER alpha	0.6	Priority exponent for sampling
PER beta (start $\rightarrow$ 1)	0.4 $\rightarrow$ 1.0	IS weight correction schedule

4.5 Checkpointing and Evaluation

Three checkpoint types are saved during training: latest.pt (every 50 episodes), best.pt (whenever the 50-episode rolling mean return improves), and final.pt (end of training). The evaluation script loads any checkpoint, sets $\varepsilon = 0$, and reports mean return, mean final position, mean pit count, best and worst finishing positions over configurable evaluation episodes. Training metrics logged per episode include: return, mean-50 return, final position, pit count, loss, ε , and replay buffer size.

4.6 Codebase Structure

The implementation is cleanly separated into six files: action_mask.py (domain rules), network.py (Dueling DQN model), replay_buffer.py (PER), dqn_agent.py (DQNConfig + DQNAgent), train_dqn.py (training loop), evaluate.py (evaluation workflow). The environment code is maintained separately under env/, with my_data.py handling all FastF1 processing.

5. Results and Analysis

5.1 Environment Validation

The F1StrategyEnv has been validated through unit tests and deterministic replay checks. Key observations from environment testing:

- A random agent (selecting uniformly from legal actions at each step) successfully runs full race episodes to termination without errors, confirming correct state transitions and episode lifecycle.
- Seeded reproducibility has been verified: identical seeds produce identical race outcomes across multiple resets.
- Safety car pit-bonus and wet-race logic are confirmed functional: pit penalties are halved under SC, and weather penalties correctly penalise mismatched tyre-to-condition pairings by up to 40 s/lap (dry tyres in wet conditions).
- The 2-compound rule penalty of -150 is correctly applied when a dry race concludes without the agent having used two distinct dry compound types.
- Driver retirement filtering ($\geq 95\%$ lap completion threshold) successfully eliminates early retirements, ensuring rich per-episode data.

5.2 DQN Training Observations

Full DQN training runs (1,500 episodes) have been initiated. While comprehensive quantitative results are still being collected, initial training observations include:

- The agent begins learning after the 2,000-step warm-up period, with measurable TD loss reduction visible in training logs.
- Episode returns show expected initial high variance, gradually stabilising as the replay buffer fills and ϵ decays.
- Action masking is observed to effectively prevent illegal strategy choices. The agent rapidly ceases attempting blocked actions even during the high-exploration phase.
- Checkpointing infrastructure is confirmed functional. `best.pt` updates are observed during training, reflecting genuine rolling-mean improvements.

Note: Formal quantitative benchmarks — convergence curves, mean final positions across evaluation episodes, and comparison against rule-based baselines — are under active collection and will be included in the final evaluation report.

5.3 DQN Evaluation Outcomes

After testing our model, we got to know about these limitations and we are currently working on these aspects or problems:

- Action masking not applied during Double DQN target computation: The next-action selection within the Double DQN target (`argmax over online_net(next_obs)`) does not apply the action mask for the next state. This can cause the target to bootstrap from actions that would be illegal at execution time, introducing inaccuracy in the learning signal. This is the highest-priority DQN fix planned.
- Replay buffer does not store action masks: Current transition storage includes (`obs`, `action`, `reward`, `next_obs`, `done`) only. Reconstructing next-state masks at training time requires either storing masks directly or re-deriving them from stored observations.
- Evaluation metrics are basic: Current evaluation reports mean return and position but lacks standard deviation, tyre choice distribution, per-track breakdowns, and comparison against rule-based or fixed-strategy baselines.
- No visualization yet: The current implementation outputs text logs only. Strategy visualization and convergence plots are planned for the next development phase.
- QRL agent not yet implemented: The hybrid VQC-DQN architecture is designed and PennyLane environment setup is in progress, but no training or comparative results are available at mid-evaluation.

5.4 DQN Training and Evaluation Process Updates (Dated: 28-06-2026)

Summary from recent training process:

```
Episode 1500 | Return -287.16 | Mean50 Return -82.39 | Final P16 | Mean50 Pos 7.36 | Pits 04 | Mean50 Pits 3.62 | Loss 0.00164 | Epsilon 0.050 | Repl
ay 87102 | Monaco 2024 STR
Saved latest checkpoint: checkpoints_v1\latest.pt
Training complete. Final model saved to: checkpoints_v1\final.pt
```

Summary from recent evaluation process:

```
Summary
-----
Checkpoint: checkpoints_v1/best.pt
Mean return: -36.749832333054556
Mean final position: 5.05
Mean pit count: 1.85
Best final position: 1
Worst final position: 14
```

```
Summary
-----
Checkpoint: checkpoints_v1/final.pt
Mean return: -83.77094228543089
Mean final position: 4.95
Mean pit count: 0.85
Best final position: 1
Worst final position: 13
PS C:\Users\mayan\Music\F1-strategy-optimization>
```

```
Summary
-----
Checkpoint: checkpoints_v1/latest.pt
Mean return: -83.77094228543089
Mean final position: 4.95
Mean pit count: 0.85
Best final position: 1
Worst final position: 13
```

6. Execution Pipeline

The end-to-end execution pipeline proceeds through the following stages. Completed stages are fully operational; planned stages have their architecture defined and will be implemented in the final phase.

Stage 1 — Data Ingestion & Caching

FastF1 API → Raw session cache (env/data/my_cache/) → F1TrackDataLoader → Processed pickle (env/data/processed_cache/) → Six typed arrays (base times, degradation, pit loss, lap grid, SC flags, wetness flags)

Stage 2 — Environment Initialisation

env.reset(seed=N) → Sample race/year + driver (retirement filter) → Initialise RaceState + RaceSession + RaceBackend → Return 15-dim observation vector

Stage 3 — DQN Training Loop

obs → action_mask() → ϵ -greedy select → env.step(action) → Store (obs, a, $r \times 0.01$, obs', done) in PER buffer → Sample batch → Compute Double DQN targets → Huber loss $\times$ IS weights → Backprop + grad clip → Update PER priorities → Soft target update → Log metrics → Checkpoint every 50 episodes

Stage 4 — Evaluation (Post-training)

Load checkpoint (best.pt / final.pt) → $\epsilon = 0$ → Run N evaluation episodes → Record return, final position, pit count, tyre sequence → Report summary statistics

Stage 5 — QRL Agent Training [Planned]

State → Classical encoder → VQC (angle encoding + CNOT ring + Rot layers) → $\langle Z_i \rangle$ expectation values → Linear readout → Q-values → Same DQN training loop on identical environment seeds

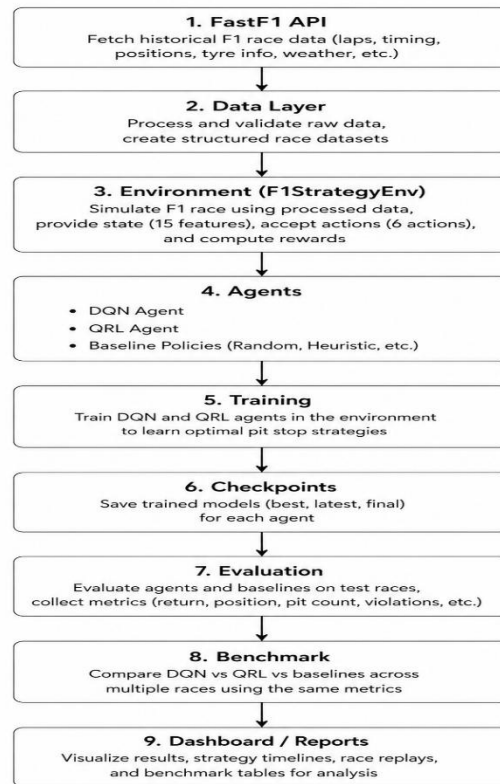
Stage 6 — Benchmarking + Dashboard [Planned]

Held-out evaluation seeds → DQN vs QRL vs rule-based vs random comparison → Streamlit + Plotly dashboard → Lap-by-lap replay + strategy divergence heatmap → Analysis report

7. Final Evaluation Deliverables (Planned)

- **Quantum RL Agent:** A fully trained hybrid VQC-DQN agent using PennyLane, evaluated on the same 9-race environment. Circuit design choices (qubit count, layer depth, entanglement topology) will be documented with justification based on training stability.
- **Benchmarking Suite:** Side-by-side comparison of DQN vs QRL vs rule-based baselines on held-out seeds. Metrics: cumulative reward, convergence rate, sample efficiency, mean final position, tyre strategy realism.
- **Streamlit Dashboard:** Lap-by-lap race replay showing QRL decisions, DQN decisions, and real historical team strategy with Plotly charts. Includes tyre compound timelines, position evolution, pit decision markers, and a strategy divergence heatmap.
- **Analysis:** Documented analysis of QRL strengths and limitations in this domain, including any observed barren plateau effects, gradient vanishing, or circuit expressibility issues.

8. GitHub & Workflow Diagram (Planned)



[Click here for the Repository.](#)

9. References and Resources

- FastF1 Library
- Gymnasium Documentation (for Environment layer)
- PyTorch Documentation (for DQN agent)
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling Network Architectures for Deep Reinforcement Learning. Proceedings of the 33rd International Conference on Machine Learning (ICML), 48, 1995–2003. (for Dueling Architecture used in DQN Agent)
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized Experience Replay. International Conference on Learning Representations (ICLR). arXiv:1511.05952. (for PER logic used in DQN Agent)
- GitHub (for version control and repository structure)
- Libraries used till now: Numpy, Pandas, PyTorch, FastF1, Gymnasium